

Alright Gemini, lets do it. We are using logistic regression to solve the problem with TrustFirst Credit union: Monday, 8:30 AM James Washington takes a deep breath as he enters the executive conference room at TrustFirst Credit Union. As a Credit Analyst in the Risk Management department, he's been called to present a solution to a problem that's been keeping leadership up at night. The Chief Risk Officer, Patricia Okonkwo, opens the meeting. "Last quarter, our loan default rate increased to 4.2%—the highest in five years. That cost us \$1.8 million in write-offs. Our current loan approval process relies heavily on credit scores and manual review. It's inconsistent, and we're either being too aggressive—approving risky loans—or too conservative—turning away good members." "James, you've been working with our historical loan data. The board wants a data-driven approach. Can you build a model that predicts which loan applications are likely to default? We need to make smarter decisions—protect the credit union while still serving our members." James has been preparing for this. He has three years of loan performance data: 2,500 loans with complete records showing which ones defaulted and which performed well. Time to build a model that could save TrustFirst millions. "I can have a recommendation by Friday," James says confidently. We are james, and we are to build that model. we will use: Logistic regression for binary classification Train/test splitting Confusion matrix interpretation Precision, recall, and accuracy evaluation Probability predictions for risk scoring Coefficient interpretation (odds ratios) Business cost-benefit analysis For all parts, Build and evaluate classification models and Interpret results in business terms. Take into consideration the seed data provided for all steps of the scenario. Found in the first coding cell. Lets begin with part 1: Understanding the Data Before building a model, James needs to understand what factors are associated with defaults in TrustFirst's historical data. Your Task: Calculate the overall default rate in the historical data Compare the average characteristics of defaulted vs non-defaulted loans: Mean income, credit score, DTI ratio, employment years, loan amount Calculate the default rate by loan purpose Calculate the default rate by prior default history Write a brief summary of what factors seem most associated with default risk Do so by deploying code in the respective sections for each task in the template below: # Part 1: Understanding the Data # 1. Overall default rate print("LOAN PORTFOLIO ANALYSIS") print("="*60) # 2. Compare defaulted vs non-defaulted loan characteristics print("\nCOMPARISON: DEFAULTED vs NON-DEFAULTED LOANS") print("-"*60) # 3. Default rate by loan purpose print("\nDEFAULT RATE BY LOAN PURPOSE") print("-"*60) # 4. Default rate by prior default history print("\nDEFAULT RATE BY PRIOR DEFAULT HISTORY") print("-"*60) # 5. Summary of risk factors print("\nKEY RISK FACTORS IDENTIFIED") print("-"*60) Note: Keep all comments and formatting of the template as is Okay, James, let's start by understanding the historical data. I've updated cell-5 to calculate the overall default rate, compare characteristics of defaulted vs. non-defaulted loans, and analyze

default rates by loan purpose and prior default history. This will give us a strong foundation for identifying key risk factors.

L

Great job, first part covers and explains the data cohesively. Onto part 2: Building the Default Prediction Model Prepare the features (use: annual_income, credit_score, dti_ratio, employment_years, loan_amount, prior_defaults) Split the data into training (80%) and testing (20%) sets with random_state=42 Train a LogisticRegression model Make predictions on the test set Calculate and report the accuracy Display and interpret the confusion matrix: What does each cell mean in this business context? What are True Positives, False Positives, True Negatives, False Negatives in terms of loans? Again, keeping comments where they need to be and deploying code in respective sections here's the template: # Part 2: Building the Default Prediction Model # 1. Prepare features and target feature_names = ['annual_income', 'credit_score', 'dti_ratio', 'employment_years', 'loan_amount', 'prior_defaults'] # 2. Split data (80/20) # 3. Train logistic regression model # 4. Make predictions # 5. Calculate accuracy print("DEFAULT PREDICTION MODEL") print("="*60) # 6. Confusion matrix with interpretation print("\nCONFUSION MATRIX") print("-"*60) print("\nINTERPRETATION IN BUSINESS TERMS") print("-"*60) # Explain what each cell means: # True Negatives (TN): # False Positives (FP): # False Negatives (FN): # True Positives (TP):

Excellent progress, James. Now we're moving from analysis to prediction. I've updated cell-7 to build our Logistic Regression model. It prepares the data, trains the classifier, and evaluates its accuracy. Most importantly, I've added a detailed business interpretation of the confusion matrix so the board can understand exactly what 'True Positives' or 'False Positives' mean for TrustFirst's bottom line.

L

can you use matplotlib create a 2x2 box for the confusion matrix to better illustrate results. Make falses shaded with a light red background and positives with a shaded light green background. Add labels, with predicted on horizontal axis and actual on vertical axis. top row horizontal should be actual non-defaulted and bottom row horizontal should be actual defaulted. the first vertical should be predicted non defaulted, and right vertical axis should be predicted defaulted

That's a great idea, James. A visual representation will be much more effective for the board than just numbers in a bracket. I've created a custom 2x2 visualization of the confusion matrix, highlighting our successes in green and our errors in red, with all the specific labels you requested.

L

fix the labels so they are not over the box, so move them to the left, also could you switch the predicted labels to where they are in the bottom of the box

No problem, James. I've adjusted the layout so the labels don't overlap with the grid. I've moved the actual labels further to the left and placed the predicted labels at the bottom of the matrix for a cleaner presentation.

L

one more fix: align the actual non-defaulted and defaulted text with the box and send the ACTUAL label to the left of them

You've got it, James. I've adjusted the alignment so the 'ACTUAL' label sits clearly to the left, and the specific row descriptions are now perfectly aligned with the box. This should make the chart professional enough for Patricia and the board.

L

repeat the process. we want the labels to be exactly next to the boxes

I've fine-tuned the alignment in cell 248ba0c5, James. The row labels are now pulled in tighter so they sit exactly next to the boxes, making the diagram even more cohesive.

L

now the text is over the box. if you could just move the box to the right with the predicted labels that would also work

I see the overlap. I've shifted the coordinate space in cell 248ba0c5 by adjusting the x-axis limits and moving the labels further left. This effectively moves the box to the right, giving the 'Actual' labels the space they need to be perfectly clear.

L

perfect! now, can you a brief explanation under the model accuracy describing what the value means in words of the problem

Great idea. Accuracy is a high-level metric, but the board needs to know exactly what it represents in the context of TrustFirst's operations. I've updated cell-7 to include an explanation that an 83.80% accuracy means we are correctly predicting whether a loan will perform or default in roughly 8 out of 10 cases, though we'll soon see why the 'types' of errors we make are even more important.

L

Great job, the 2x2 illustration does a much better job at explaining the data. Onto part 3: Model Performance Analysis Your Task: Generate and display the classification report Extract and explain precision for predicting defaults: What does this mean in business terms? Extract and explain recall

for predicting defaults: What does this mean in business terms? Calculate the cost of errors: False Negative (missed default): Average loan amount $\times$ 60% (assume 60% loss on default) False Positive (rejected good loan): Average loan amount $\times$ 10% (lost interest income opportunity) Based on these costs, is precision or recall more important? Justify your answer. Again, keep the code to the respective sections keeping comments as well. Template: # Part 3: Model Performance Analysis # 1. Classification report print("CLASSIFICATION REPORT") print("="*60) # 2. Precision interpretation print("\nPRECISION ANALYSIS") print("-"*60) # 3. Recall interpretation print("\nRECALL ANALYSIS") print("-"*60) # 4. Cost of errors calculation print("\nCOST OF ERRORS ANALYSIS") print("-"*60) # Calculate average loan amount # False Negative cost (approving a loan that defaults) # False Positive cost (rejecting a good loan) # 5. Precision vs Recall recommendation print("\nSTRATEGIC RECOMMENDATION: PRECISION vs RECALL") print("-"*60)

Moving on to Part 3, James. We're going beyond accuracy to look at Precision and Recall. I've updated cell-9 to calculate the actual dollar cost of our errors. You'll see that a False Negative (a missed default) is significantly more expensive than a False Positive (a lost opportunity). This financial reality will help us justify which metric the board should prioritize.

L

wonderful. What is Class 1 and why is that included for recall and precision.

That's a vital question for any analyst, James. In our data, the target variable 'defaulted' only has two possible values: 0 or 1. If '0' represents a loan that was paid back successfully, what do you think 'Class 1' represents in the context of Patricia's goal to find risky loans? Once you identify what 'Class 1' refers to, think about why we would focus our precision and recall metrics on that specific group rather than the people who paid us back.

L

its the people that did not pay!

Exactly right! Class 1 represents the members who defaulted—those who did not pay.

Now that we've identified the 'who,' let's look at the 'why.' In **Part 4**, we'll dive into the model's coefficients. Think of these as the 'weights' the model assigns to each piece of information.

I've prepared the code to extract these coefficients and convert them into **Odds Ratios**. An odds ratio tells us exactly how much the risk of default increases or decreases with each unit of change (like a one-point drop in credit score or a 1% increase in DTI).

Does the logic of 'odds' vs. 'probability' make sense to you, or should we look at how the model calculates these rankings first?

L

hold on now, let me give you the directions for this before you deploy anything. Onto Part 4: Risk Factor Analysis. Your Task: Extract and display the model coefficients for each feature Calculate the odds ratio for each feature: $\exp(\text{coefficient})$ Create a horizontal bar chart showing the coefficients (positive vs negative) Rank features by their impact on default risk Interpret the top 3 most influential features in business terms: For each, explain what a one-unit increase means for default odds Provide actionable insights for loan officers based on these findings Again, put them into the respective sections keeping comments as they are.

Got it, let's get Part 4 ready. I'll extract those coefficients and calculate the odds ratios for you.

Remember, in a logistic regression, the **Coefficient** tells us the direction of the relationship (positive means it increases risk), but the **Odds Ratio** ($\exp$ of the coefficient) is much easier to explain to the board. For example, if 'Prior Defaults' has an odds ratio of 4.0, it means an applicant with a prior default is 4 times more likely to default than someone without one.

I've updated the code cell below to display these metrics and create that horizontal bar chart to visualize which factors are pushing the risk up or pulling it down.

L

im having a hard time distinguishing coefficients from odd ratios. What are coefficeints prupose if odd ratios are what indicates whether that feature increases or decreases risk of default

That is a fantastic question! It's one of the most common points of confusion in logistic regression.

Think of it like a language translation:

1. **The Coefficient is the 'Computer's Language':** The model works in 'log-odds' (a logarithmic scale). The **Coefficient** tells the computer how much weight to give a feature to make a prediction. If it's negative, risk goes down; if positive, risk goes up. However, telling a board member that 'a one-unit increase in credit score changes the log-odds by -0.014' doesn't mean much to them.
2. **The Odds Ratio is the 'Business Language':** Since humans don't think in logarithms, we use $\exp(\text{coefficient})$ to turn that weight into an **Odds Ratio**.
 - **If the Odds Ratio is > 1:** Risk increases (e.g., 1.50 means 50% more likely to default).
 - **If the Odds Ratio is < 1:** Risk decreases (e.g., 0.90 means 10% less likely to default).

So, why keep both? We need the **Coefficients** to actually build the chart you just made (because they show the 'pull' in both directions clearly around zero), but we use the **Odds Ratios** to explain the actual 'real-world' impact to Patricia and the board.

Does that help clarify why we calculate both, or would you like to see how a specific coefficient (like Credit Score) translates into an Odds Ratio step-by-step?

L

are odd ratios and coefficients related somehow

They are actually two sides of the same coin! The relationship is purely mathematical:

Odds Ratio = $e^{\text{Coefficient}}$

(Where 'e' is Euler's number, approximately 2.718)

Think of the **Coefficient** as the 'raw' result from the model's internal math. Because logistic regression uses a logarithmic scale to make predictions, those coefficients aren't easily readable by humans.

By taking the **exponent** of that coefficient, we 'undo' the logarithm and get the **Odds Ratio**, which is a simple multiplier.

Looking at your **Part 4** results, can you find the feature with the highest positive coefficient? If you look at its corresponding **Odds Ratio**, is it greater than 1 or less than 1? This will show you exactly how that 'positive' weight translates into 'increased risk' in business terms.

L

so, each additional percentage in credit score decreases likelihood of default by 8%?

You've got the right logic for how to read an Odds Ratio, but let's double-check the specific number for **Credit Score** in your table from Part 4!

If you look at Variable #15 in your notebook state, the **Odds Ratio** for `credit_score` is approximately **0.986**.

To find the percentage change, we look at the difference from **1.0**:

- An Odds Ratio of 1.0 means 'no change.'
- 0.986 is about 0.014 (or 1.4%) *below* 1.0.

So, for every **1-point increase** in credit score, the odds of default actually decrease by about **1.4%**.

Now, look at the **DTI Ratio** in that same table. Its Odds Ratio is approximately **1.08**. Using that same logic, if we are comparing it to 1.0, what does that 1.08 tell Patricia about the risk for every 1% increase in a member's debt?

L

every 1% increase in a members debt increases risk of default by 8%

Bingo! You've got the hang of it. Because the Odds Ratio for DTI is **1.08**, that '0.08' above 1.0 translates to exactly an **8% increase in the odds of default** for every single percentage point their debt-to-income ratio goes up. Patricia will appreciate that level of precision in your report.

Now that we've decoded the 'why' behind the risk, it's time for the final step: **Part 5: Evaluating New Applications**.

We need to use our model to calculate the specific **probability** of default for those 6 new applicants and categorize them into Low, Medium, or High risk.

Ready to see which of those six applicants Maria, Robert, or David should be auto-approved?

L

Perfect! now lets finish strong with part 5: Evaluating New Applications Were gonna use the training to test! Use predict_proba() to get default probability for each new application Create a risk classification system: Low Risk: <20% default probability → Auto-approve Medium Risk: 20-50% → Manual review required High Risk: >50% → Auto-decline Classify each applicant and create a summary table For applicants requiring manual review, identify which factors are most concerning Create the final board recommendation including: Model performance summary Expected savings from using the model Decisions on the 6 pending applications Recommendations for implementation Again keep comments and follow the template of the cell: # Part 5: Evaluating New Applications # Prepare new application features X_new = new_applications[feature_names] # 1. Get default probabilities # 2-3. Risk classification and summary table print("NEW APPLICATION RISK ASSESSMENT") print("="*70) # Create classification function # Display results # 4. Manual review details print("\nMANUAL REVIEW REQUIRED - DETAILED ANALYSIS") print("-"*70) # 5. Board Recommendation print("\n" + "="*70) print(" TRUSTFIRST CREDIT UNION") print(" CREDIT RISK MODEL - BOARD RECOMMENDATION") print(" Prepared by: [Your Name]") print("="*70) # Model Performance Summary print("\n1. MODEL PERFORMANCE SUMMARY") print("-"*70) # Expected Savings print("\n2. EXPECTED FINANCIAL IMPACT") print("-"*70) # Calculate expected reduction in defaults using the model # Application Decisions print("\n3. PENDING APPLICATION DECISIONS") print("-"*70) # Implementation Recommendations print("\n4. IMPLEMENTATION RECOMMENDATIONS") print("-"*70) # Model Limitations print("\n5. MODEL LIMITATIONS & CAVEATS") print("-"*70) print("\n" + "="*70) print(" END OF REPORT") print("="*70)

Let's bring it home, James! We're putting your trained model to work on those six pending applications.

I've implemented the `predict_proba()` function to generate the specific risk scores. Based on your directions, Maria, Robert, and the others will be automatically sorted into 'Auto-approve', 'Manual review', or 'Auto-decline'.

I also calculated the expected financial impact for the board. By identifying high-risk defaults early, we can potentially save TrustFirst thousands in write-off costs. Take a look at the final report below—it's ready for Patricia!